



hbnb

Documentation Compilation

Robin Allix and Lucas Nevano

For **HOLBERTON** School

High-Level Package Diagram – HBnB Application

1.INTRODUCTION

The objective of this exercise is to design a high-level package diagram that represents the overall architecture of the HBnB application.

This diagram provides a conceptual overview of how the application is structured, how its main components are organized, and how they interact with each other.

The architecture follows a layered design approach, separating responsibilities into distinct layers to improve clarity, maintainability, and scalability.

The communication between layers is simplified using the Facade design pattern, which ensures loose coupling and a clean separation of concerns.

This documentation describes the role of each layer, the interactions between them, and the purpose of the key architectural elements represented in the UML package diagram.

2.HIGH-LEVEL ARCHITECTURE

Layered Architecture Overview

The HBnB application is organized into three main layers, each with a clearly defined responsibility.

Each layer performs exactly one role and communicates only with the appropriate adjacent layer.

Request Flow Overview

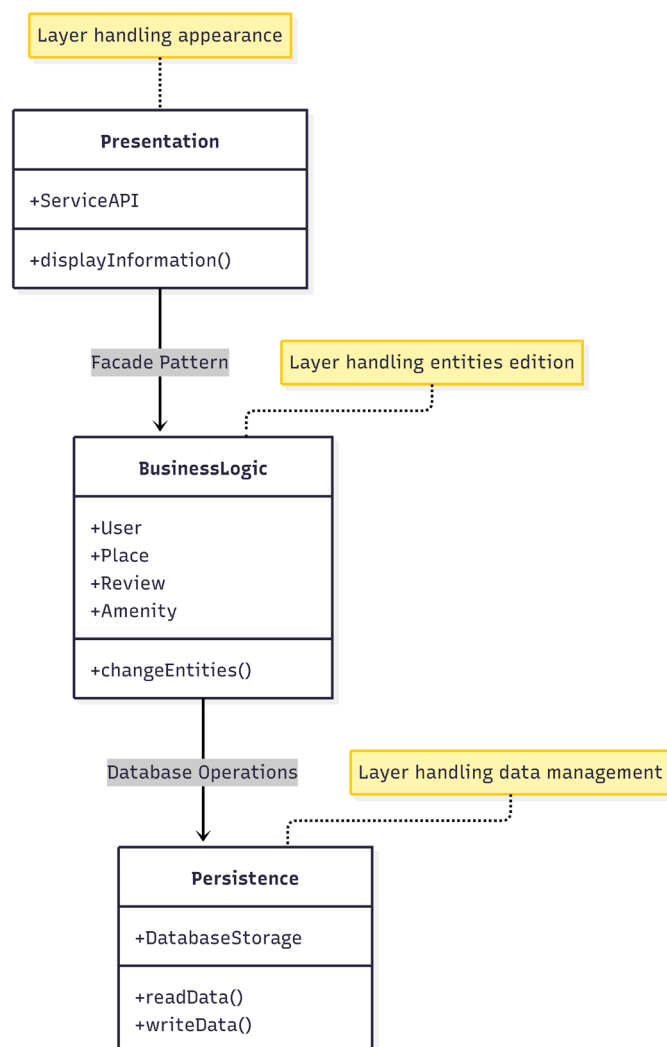
1. The Presentation Layer receives the user request.
2. The Facade forwards the request to the Business Logic Layer.
3. The Business Logic Layer applies business rules and prepares any changes.
4. The Persistence Layer reads or writes data as requested.
5. The result is returned to the Business Logic Layer.
6. The Presentation Layer sends the response back to the user.

This strict flow ensures that responsibilities remain well separated and that the system remains easy to understand and evolve.

High-Level Package Diagram

The package diagram visually represents:

- The three architectural layers.
- The components contained in each layer.
- The communication pathways between layers.
- The use of the Facade pattern as intermediary.



Presentation Layer (Services, API)

This layer handles the interaction between the user and the application. It includes all the services and APIs that are exposed to users.

- Acts as the entry point to the application.
- Handles incoming requests.
- Outgoing responses.
- Defines how data is exposed and formatted.

The Presentation Layer does not contain any business logic and does not interact directly with the database. Its role is strictly limited to communication.

Service API

The ServiceAPI represents all exposed API services and endpoints. These endpoints receive requests, extract the required input data, and delegate processing to the facade.

displayInformation()

It represents the response-handling logic of the Presentation Layer.

- Receives the result from Business Logic Layer.
- Formats the output appropriately.
- Sends the response back to the client.

Facade Pattern

The Facade Pattern is positioned between the Presentation Layer and the Business Logic Layer.

- Acts as a single point of contact for the presentation layer.
- Simplifies access to business operations.
- Hides internal complexity by exposing a limited and clear interface.

Instead of interacting directly with multiple entities, the Presentation Layer communicates only with the facade, reducing coupling and improving maintainability.

3. BUSINESS LOGIC LAYER (MODELS)

This layer contains the core business logic and the models that represent the entities in the system (e.g., User, Place, Review, Amenity).

- Applies business rules.
- Ensures data consistency and validity.
- How entities are created, modified, or deleted.

Domain Entities

BaseModel : Parent class shared by all others, provides unique ID and timestamps (created_at, updated_at)

Attributes

Id : identification of the object
CreatedAt : datetime of the creation
UpdatedAt : datetime of the updating

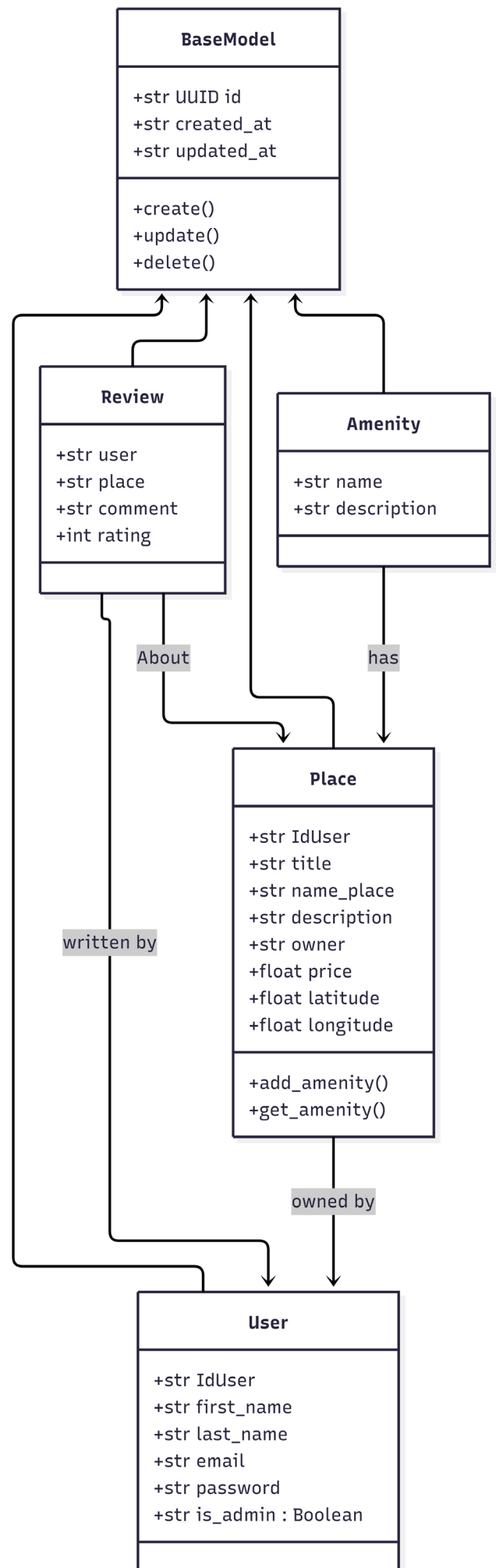
Methods

create() to create data
update() to update data
delete() to delete data

User : A person using the app, can own places and write review.

Attributes

IdUser : identification of the user
First_name : identify of the user
Last_name : identify of the user
Email : connexion information
Password : connexion information
Is_admin : administrator status (boolean)



Place : A listing created by a user, includes location, price, number of rooms, etc...

Attributes

IdUser : key of class user to join it
 Title : title of the place
 Name_place : name of the place
 Description : description of the place
 Owner : owner of the place
 Price : cost of the place
 Latitude : coordonate of the place
 Longitude : coordonate of the place

Methods

add_amenity() to add amenity to the place
 get_amenity() to get amenity to the place

Review : A feature(e.g, WI-FI, pool, extra accessories) that can be addes to a place.

Attributes

User : associated with the user
 Place : associated with the place
 Comment : commentary user
 Rating : notation user

Amenity : Feedback written by a user about a specific place.

Attributes

Name : name of amenity
 Description : description of amenity

changeEntities()

It represents all operations that modify the state of an entity.

- Validates business rules.
- Determines whether an operation is allowed.
- Prepares data to be persisted.

The Business Logic Layer does not perform database operations directly. Instead, it delegates data storage to the Persistence Layer.

Database Operations

Database Operations represent the interaction between the Business Logic Layer and the Persistence Layer. When the Business Logic Layer needs to store, retrieve, or modify data, it delegates these operations to the Persistence Layer.

Persistence Layer

The Persistence Layer is responsible for managing data storage and retrieval. It does not apply business rules and does not make decisions.

- Interacts directly with the database or storage mechanism.
- Executes read and write operations.
- Ensures data durability.

DatabaseStorage

DatabaseStorage represents the underlying storage mechanism. This abstraction allows the storage technology to change without impacting the Business Logic Layer.

- A database.
- A file system.
- An ORM-based storage solution.

readData()

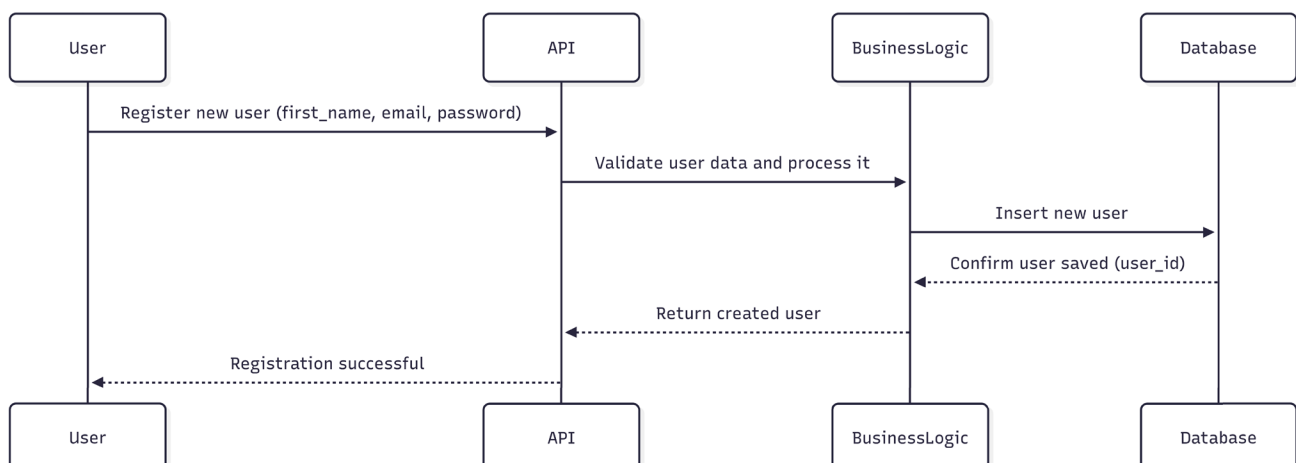
It retrieves stored data and performs simple data access operations without validation or interpretation.

writeData()

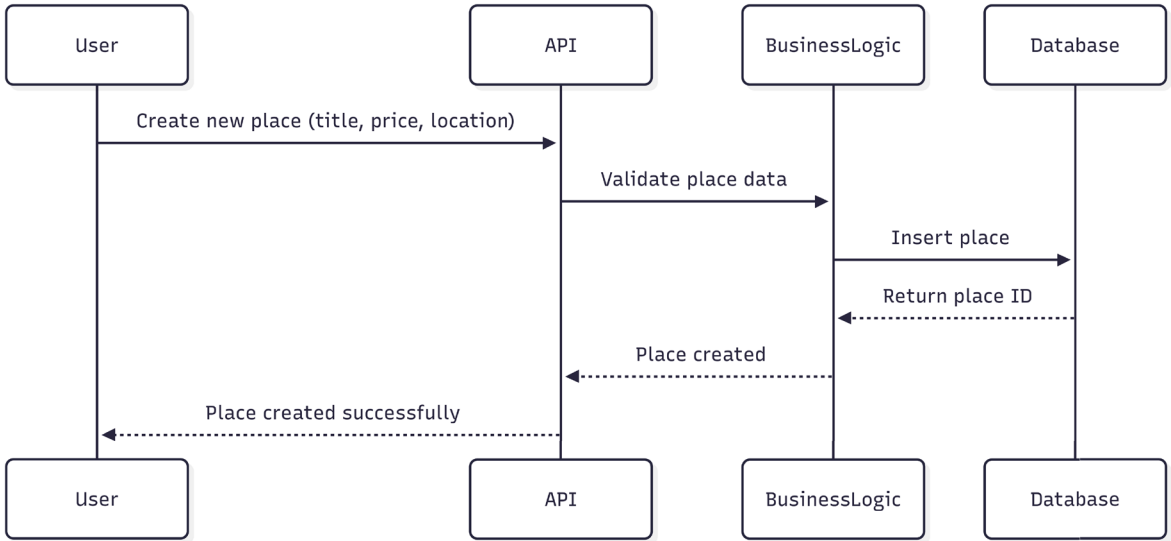
It stores new data or updates existing data and assumes that all business decisions have already been made.

4. API INTERACTION FLOW

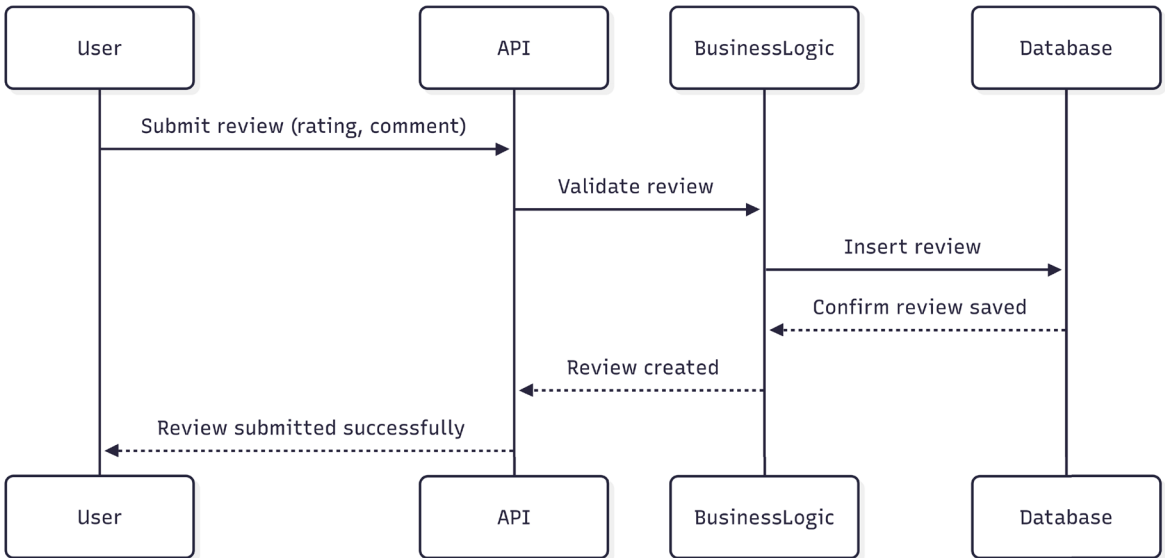
User registration



Place Creation



Review Submission



Fetching Places

